

Assignment

Choose one assignment below, complete it fully, and submit. Completing both assignments is entirely optional, but candidates who do will be given strong preference in our evaluation. A clean, well-explained single-track submission always beats a rushed attempt at both.

Backend Assignment

Build the REST API + async worker that powers Closira's enquiry-handling pipeline.

Frontend Assignment

Build the mobile dashboard that business owners use to monitor conversations and act on escalations.

About Closira

Closira is an AI-powered customer communication platform built for small and medium businesses (SMBs). It handles inbound customer enquiries across WhatsApp, email, and phone, responding using business-defined SOPs to qualify leads, schedule follow-ups, and escalate to human agents when required.

As an engineering intern, you will be building the systems that power these workflows: APIs, async jobs, and the data and UI layers that tie it all together.

1. Backend Assignment Details

1.1 Objective

Build a lightweight backend service that simulates Closira's core customer enquiry-handling workflow. We want to evaluate your backend fundamentals, API design thinking, async processing understanding, and engineering ownership.

***Note:** We are not looking for a production-ready system. We want to see how you think, how you structure a backend, and whether you take ownership of your decisions. A clean, well-explained partial submission is valued over a messy but complete one.*

1.2 What You Will Build

A REST API backend in Python + FastAPI that handles inbound customer enquiries, processes them asynchronously, and exposes status and history endpoints.

1.3 API Endpoints

Method	Endpoint	Description
--------	----------	-------------

POST	/enquiry	Create a new inbound customer enquiry. Accept channel (WhatsApp / email/call), customer name, and message. Return a job ID immediately; do not block.
POST	/enquiry/{id}/follow-up	Schedule a follow-up for an open enquiry. Accept a delay in minutes and an optional message template.
POST	/enquiry/{id}/escalate	Mark an enquiry as escalated to a human agent. Accept a reason field. Update the status in the database.
GET	/enquiry/{id}/history	Return the full conversation history and status timeline for a given enquiry.
GET	/health	Return API status and database connectivity as a simple health check.

1.4 Async Background Task

After a new enquiry is created, trigger a background task (using Celery or FastAPI BackgroundTasks, your choice) that:

- Matches the inbound message to one of 3–5 hardcoded SOPs (e.g., booking enquiry, pricing question, complaint, after-hours message) using basic keyword logic, no AI required
- Updates the enquiry record with the matched SOP and a suggested response
- If no SOP matches, flags the enquiry as escalated and logs the event

Justify your choice of Celery vs FastAPI BackgroundTasks briefly in your README.

1.5 Data & Logging

- Store enquiry data using PostgreSQL or SQLite (your choice, explain it in the README)
- Add structured logging in JSON format for key events: enquiry created, task processed, SOP matched, escalation triggered
- Handle errors gracefully, meaningful HTTP status codes, and no unhandled exceptions reaching the client
- FastAPI auto-docs (/docs) must be complete with descriptions and example payloads

1.6 Backend Assignment Deliverables

GitHub repository link (public or shared). Clean commit history is a strong positive signal.

- README covering: setup and run instructions, database schema and reasoning, Celery vs BackgroundTasks decision, trade-offs or known limitations
- API test file, Postman collection, curl commands, or a .http file covering all five endpoints with example payloads

- 2–5 minute video walkthrough explaining your prototype

1.7 Evaluation Criteria for Backend Assignment

Criterion	What we look for
Code quality & structure	Clean, readable, well-organised code. Sensible module separation. No unnecessary complexity.
API design clarity	Intuitive routes, correct HTTP methods, useful error messages, complete /docs.
Scalability thinking	Schema design, async processing choice, tenant-awareness (even if stubbed).
Reliability & error handling	Edge cases handled, no crashes on bad input, meaningful status codes.
Documentation quality	The README is clear enough for a new team member to set up and run without asking questions.
Engineering ownership	Decision rationale explained, trade-offs acknowledged, not just code, but thinking.

2. Frontend Assignment Details

2.1 Objective

Design and build a mobile dashboard experience in React Native that gives a business owner a clear, actionable view of their customer communication activity. We are evaluating your UI quality, component thinking, and UX clarity.

***Note:** Do not spend time building a backend, authentication, or real API calls. We are evaluating your mobile UI skills, not your ability to integrate services. A polished, well-structured frontend will always score higher than a feature-heavy but messy one.*

2.2 What You Will Build

A React Native mobile app with bottom tab navigation. All data should be hardcoded as mock JSON, no backend required.

2.3 Required Screens

Screen	Must Include
--------	--------------

Dashboard (Home)	Summary stats: total leads today, missed enquiries, open escalations, follow-ups due. Quick-action buttons. Activity feed showing recent conversations.
Leads	List of inbound leads with channel badge (WhatsApp / Email / Call), status (new/qualified / escalated), customer name, and time received. Tappable lead card.
Escalations	Active escalation alerts with reason, customer name, channel, and time. Urgency indicator (high / medium). Resolve button per card.
Follow-ups	Task cards for scheduled follow-ups: customer name, due time, message preview, and mark-as-done action.
Conversation Detail	Tapping a lead or escalation opens a conversation view: message thread, SOP match label, AI summary, and status timeline.

2.4 Navigation Structure

Implement a bottom tab navigator with icons and labels for: Home, Leads, Escalations, and Follow-ups. The Conversation Detail screen should open as a stack screen from Leads or Escalations.

2.5 Mock Data

Hardcode realistic mock data in a /mock folder. Structure it as if it were coming from an API JSON objects with proper field names, not inline strings. We will look at this to understand your API-readiness thinking.

```
{ "id": "enq_001", "customer": "Sarah M.", "channel": "whatsapp",
  "status": "escalated", "reason": "Pricing complaint",
  "receivedAt": "2025-05-20T09:14:00Z",
  "summary": "Customer unhappy with quote, requested manager." }
```

2.6 UI & Code Expectations

- Component-based: each section of the UI should be its own reusable component. No monolithic screen files.
- Styling: use NativeWind (Tailwind for React Native) or StyleSheet, justify your choice in the README.
- Consistent spacing scale, readable font sizes, clear visual hierarchy throughout.
- Channel badges: WhatsApp (green), Email (blue), Call (amber), consistent across all screens.

- Status indicators: New (blue), Qualified (green), Escalated (red), clear and consistent.
- Empty states: handle empty lists gracefully with a helpful message, not a blank screen.

2.7 Frontend Assignment Deliverables

GitHub repo (public or shared), clean folder structure, components separated, mock data in /mock

- README covering: setup and run instructions, styling choice reasoning, screenshots or screen recordings, and known limitations
- Screenshots or screen recordings of every screen
- 2–5 minute video walkthrough explaining your prototype

2.8 Evaluation Criteria for Frontend Assignment

Criterion	What we look for
UI quality	Does it look like a real product? Clean layout, consistent colours, readable typography, no broken spacing.
Code structure	Components are reusable and well-named, no giant monolithic files. Folder structure makes sense.
UX clarity	Is it intuitive? Can a business owner understand the dashboard without explanation?
Attention to detail	Badges, empty states, status indicators, edge cases, and the small things show the big thinking.
Product thinking	Does the UI reflect an understanding of what Closira does and who uses it? Or does it feel generic?

Bonus: Complete Both Assignments

Submitting both the Backend Assignment and Frontend Assignment is entirely optional. There is no penalty for choosing only one. However, candidates who submit strong work across both tracks will be given clear preference during evaluation, as it demonstrates full-stack capability and genuine interest in the role.

If you attempt both: submit a single GitHub repository with clearly separated folders (/backend and /frontend), a combined README, and one video walkthrough covering both.

General Submission Notes

- Submit your GitHub repository link before the 48-hour deadline.

- Your repository should have a clean commit history; this is a positive signal about how you work.
- We review documentation seriously. A README that lets a new engineer run your project without asking questions is a strong submission.
- If you make a trade-off or leave something incomplete, note it in the README. Owning your decisions matters more than pretending they don't exist.

Questions? Reach out to us.